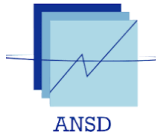


République du Sénégal

Ministère de l'économie, du Plan
et de la Coopération



Agence nationale de la Statistique
et de la Démographie



Ecole nationale de la Statistique
et de l'Analyse économique Pierre
NDIAYE

Pipeline de Traitement des Données de l'Enquête sur la Gestion des Ordures Ménagères

.....

*Documentation Technique Complète — Traitement, Analyse et
Automatisation des Données
Région de Dakar — 2018*

Présenté par :

- Josée Clémence JEAZE NGUEMEZI
 - David Christ MEKONTCOU NZONDE
 - Ndeye Khary SALL
 - Fatou Soumaya WADE
- Élèves Analystes Statisticiens — Option
Data Science, 3^e année*

Sous la supervision de :

M. Mamadou MBODJ
Ingénieur Statisticien Économiste

Année académique : 2025–2026

Table des matières

Avertissement	6
Remerciements	7
Sigles et acronymes	8

I PRÉSENTATION ET ARCHITECTURE DU PIPELINE

9

1 Présentation du projet	10
I Objectifs	10
II Résultats du traitement	10
2 Architecture du pipeline	12
I Séquence d'exécution	12
II Diagramme de flux	12
III Principe de scalabilité	12
3 Structure des fichiers	13

II DOCUMENTATION TECHNIQUE DES MODULES 14

4 Module config.py — Configuration centrale	15
I Chemins	15
II Mappings Module I — Ménage et CM	16
III Mappings Module II — Déchets	17
IV Proxy montant mensuel FCFA	17
V Seuils de contrôle qualité	18
5 Module loader.py — Chargement et contrôle qualité	19
I Fonctions principales	19
II Contrôles de cohérence	20

III	Code source	20
6	Module builder.py — Construction des variables	23
I	Helpers internes	23
II	Bloc A — Ménage (12 variables)	24
III	Bloc B — Chef de ménage (19 variables)	25
IV	Bloc C — Déchets ménagers (87 variables)	25
IV.1	Sources et natures (II.1–II.2)	25
IV.2	Stockage (II.3–II.6)	26
IV.3	Tri et revente (II.7–II.10)	26
IV.4	Traitement (II.11–II.12)	26
IV.5	Bennes tasseuses — service public (II.13–II.20)	26
IV.6	Service alternatif — entités privées (II.21–II.23)	26
IV.7	Paiement (II.23–II.25)	26
IV.8	Infrastructure quartier (III.1–III.33)	26
V	Bloc D — Conséquences sanitaires (14 variables)	27
7	Modules exporter.py et qaqc.py	29
I	exporter.py	29
II	qaqc.py — Rapport Excel 6 onglets	30
8	Orchestrateur run_pipeline.py	31
9	API FastAPI — main.py	33
I	Routes	33
II	Flux POST /api/process	33
III	Code source	33
III	VARIABLES, UTILISATION ET DÉPLOIEMENT	36
10	Variables produites et absentes	37
I	Récapitulatif	37
II	Variables absentes	37
11	Guide d'utilisation et déploiement	38
I	Installation	38
II	Option A — Ligne de commande	38
III	Option B — Plateforme web locale	38
IV	Déploiement sur Render	38

V	Dépendances Python	39
---	------------------------------	----

Liste des tableaux

1.1	Indicateurs clés du pipeline	10
2.1	Modules du pipeline	12
4.1	Mappings du Module I	16
4.2	Mappings du Module II	17
4.3	Calcul du proxy montant mensuel	17
4.4	Seuils QC	18
5.1	Fonctions de loader.py	19
5.2	Contrôles de cohérence appliqués	20
6.1	Variables du Bloc A	24
6.2	Variables du Bloc B	25
6.3	Variables du Bloc D	27
7.1	Fonctions de exporter.py	29
7.2	Structure du rapport QAQC	30
9.1	Routes de l'API	33
10.1	Variables analytiques produites	37
10.2	Variables absentes et documentation	37
11.1	requirements.txt	39

Avertissement

« Les points de vue émis dans ce document sont propres à ses auteurs. L'École Nationale de la Statistique et de l'Analyse Économique (ENSAE) Pierre NDIAYE et l'Unité de Coordination de la Gestion des Déchets Solides (UCG) n'entendent donner aucune approbation ni improbation. »

Remerciements

Nous tenons à exprimer notre profonde gratitude à M. Mamadou MBODJ, Ingénieur Statisticien Économiste, pour son encadrement rigoureux, sa disponibilité et ses conseils avisés tout au long de la réalisation de ce pipeline de traitement de données. Nos remerciements vont également à l'ensemble du corps professoral de l'ENSAE Pierre NDIAYE pour la qualité de la formation reçue, ainsi qu'à l'Unité de Coordination de la Gestion des Déchets Solides (UCG) pour le partage des données de l'enquête sur la gestion des ordures ménagères dans la région de Dakar.

Sigles et acronymes

ANSD : Agence Nationale de la Statistique et de la Démographie.

API : Interface de Programmation Applicative (*Application Programming Interface*).

ASGI : *Asynchronous Server Gateway Interface*.

CM : Chef de Ménage.

CSV : *Comma-Separated Values*.

DTA : Format de fichier natif du logiciel Stata.

ENSAE : École Nationale de la Statistique et de l'Analyse Économique Pierre Ndiaye.

FCFA : Franc de la Communauté Financière Africaine.

QAQC : Contrôle et assurance qualité (*Quality Assurance / Quality Control*).

REST : *Representational State Transfer*.

UCG : Unité de Coordination de la Gestion des Déchets Solides.

XLSX : Format de fichier tableur Microsoft Excel.

Première partie

PRÉSENTATION ET ARCHITECTURE DU PIPELINE

Chapitre 1

Présentation du projet

Ce projet implémente un **pipeline de traitement de données reproductible et scalable** pour l'enquête sur la gestion des ordures ménagères dans la région de Dakar, menée conjointement par l'ENSAE et l'UCG en 2018. Il couvre **2 474 ménages** répartis dans les départements de Dakar, Pikine, Rufisque et Guédiawaye.

I Objectifs

- Extraire les données brutes au format Stata (`.dta`) depuis un dossier `input/`
- Nettoyer et valider les données (valeurs aberrantes, incohérences, données manquantes)
- Construire **132 variables analytiques** organisées en 4 blocs thématiques
- Exporter la table consolidée en CSV et Stata (`.dta`)
- Produire un rapport de contrôle qualité Excel (QAQC) en 6 onglets
- Exposer l'ensemble via une API REST (FastAPI) avec interface web

II Résultats du traitement

Indicateur	Valeur
Ménages traités	2 474
Variables brutes	139
Variables analytiques	132
Anomalies QC détectées	49
Durée d'exécution	≈ 2 secondes
Fichiers produits	3 (CSV, DTA, XLSX)

TABLE 1.1 – Indicateurs clés du pipeline

Le pipeline tourne en **2 secondes** sur l'ensemble des 2 474 ménages, sans erreur, et produit 3 fichiers de sortie prêts à l'emploi.

Chapitre 2

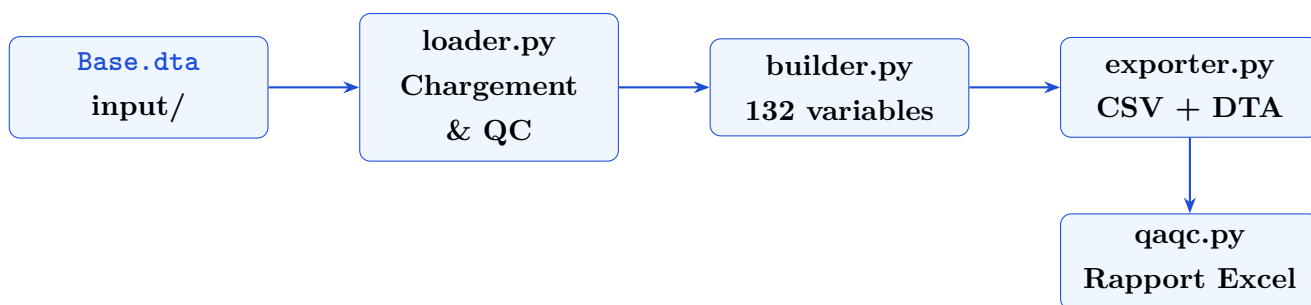
Architecture du pipeline

I Séquence d'exécution

Étape	Module	Rôle
1	<code>loader.py</code>	Chargement du .dta, nettoyage, contrôles qualité
2	<code>builder.py</code>	Construction des 132 variables analytiques
3	<code>exporter.py</code>	Export CSV (UTF-8 BOM) et Stata .dta
4	<code>qaqc.py</code>	Rapport Excel 6 onglets

TABLE 2.1 – Modules du pipeline

II Diagramme de flux



III Principe de scalabilité

Toute la logique métier est centralisée dans `config.py`. Pour une nouvelle édition de l'enquête :

1. Mettre à jour `config.py` si les codes ont changé
2. Relancer `python run_pipeline.py`
3. **Aucune modification** des autres modules n'est nécessaire

Chapitre 3

Structure des fichiers

```
Groupe_Insalubrite_traitement_donnees/  
|-- input/                                <- Donnees brutes (non versionnees)  
|   '-- Base.dta  
|-- output/                              <- Resultats generes automatiquement  
|   |-- dataset_dechets.csv  
|   |-- dataset_dechets.dta  
|   |-- QAQC_rapport.xlsx  
|   '-- pipeline_YYYYMMDD.log  
|-- src/                                  <- Code source du pipeline  
|   |-- config.py                        <- Mappings et parametres centralises  
|   |-- loader.py                       <- Chargement et controles qualite  
|   |-- builder.py                      <- Construction des variables  
|   |-- qaqc.py                         <- Rapport qualite Excel  
|   |-- exporter.py                    <- Export CSV et Stata  
|   '-- run_pipeline.py                 <- Orchestrateur principal  
|-- templates/  
|   '-- index.html                      <- Interface web  
|-- main.py                             <- API FastAPI  
|-- requirements.txt  
|-- render.yaml  
'-- README.md
```

Listing 3.1 – Arborescence du projet

Note

Les dossiers `input/` et `output/` sont dans `.gitignore` — les données ne sont pas versionnées sur GitHub.

Deuxième partie

DOCUMENTATION TECHNIQUE DES MODULES

Chapitre 4

Module config.py — Configuration centrale

Ce fichier centralise **tous** les mappings de codes, labels et paramètres. C'est le seul fichier à modifier entre deux éditions.

I Chemins

```
1 BASE_DIR      = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
2 INPUT_DIR     = os.path.join(BASE_DIR, "input")
3 OUTPUT_DIR    = os.path.join(BASE_DIR, "output")
4 INPUT_FILE    = "Base.dta"
```

II Mappings Module I — Ménage et CM

Variable	Codes et labels
SEXE	1 → Homme 2 → Femme
STATUT_REPONDANT (I.4)	1→CM 2→Femme/Mari 3→Fils/Fille 4→Gendre ... 10→Employé
STATUT_OCCUPATION (I.6)	1→Locataire 2→Propriétaire 3→Logé par un tiers
TYPE_LOGEMENT (I.7)	1→Case/Baraque 2→Maison basse 3→Maison en étage 4→Appartement
TYPE_ENSEIGNEMENT_CM (I.10)	1→Française 2→Arabe 3→Coranique 4→Aucun 5→NSP
NIVEAU_INSTRUCTION_CM (I.11)	1→Primaire 2→Secondaire 3→Supérieur 4→NSP
ALPHA (I.12)	1→Oui 2→Non 3→Ne sait pas

TABLE 4.1 – Mappings du Module I

III Mappings Module II — Déchets

Variable	Codes et labels
MODE_STOCKAGE (II.3)	1→Poubelle 2→Seau/Bassine 3→Fût 4→Carton 5→Sac
PLACE_STOCKAGE (II.5)	1→Cuisine 2→Parcelle 3→Rue proche 4→Rue loin
ACCES_BENNE (II.13)	1→Oui 2→Non
DISTANCE_COLLECTE (II.16)	1→Proche 2→Acceptable 3→Éloignée
FREQUENCE_BENNE (II.19)	1→Régulière 2→Pas régulière
SATISFACTION_BENNE (II.20)	1→Très bon 2→Bon 3→Mauvais 4→Très mauvais
SERVICE_ALTERNATIF (II.22)	1→Charrettes 2→Société pré-collecte 3→Associations
MONTANT_EVACUATION (II.25_1)	1→<200 F 2→200-400 F 3→401-1000 F 4→1001-2000 F 5→>2000 F
FREQUENCE_PAIEMENT (II.25_2)	1→Jour 2→Semaine 3→Mois 4→An

TABLE 4.2 – Mappings du Module II

IV Proxy montant mensuel FCFA

$$\text{montant_mensuel} = \text{médiane}(\text{tranche}) \times \text{facteur}(\text{fréquence})$$

Tranche	Médiane (FCFA)	Facteur mensuel	Exemple
1 (< 200 F)	100	×1	100 F/mois
2 (200–400 F)	300	×4,33	1 299 F/mois
3 (401–1000 F)	700	×30	21 000 F/mois
4 (1001–2000 F)	1 500	÷12	125 F/mois
5 (> 2000 F)	2 500	selon fréq.	—

TABLE 4.3 – Calcul du proxy montant mensuel

Note

Si `II_24` $\neq$ 1 (service non payant), le montant est mis à **0 FCFA**.

V Seuils de contrôle qualité

Paramètre	Valeur	Application
<code>QC_AGE_MIN</code>	15 ans	Âge minimum réaliste CM
<code>QC_AGE_MAX</code>	110 ans	Âge maximum réaliste
<code>QC_TAILLE_MAX</code>	50 personnes	Taille ménage maximale
<code>QC_MISSING_THRESH</code>	50%	Alerte si taux dépasse ce seuil

TABLE 4.4 – Seuils QC

Chapitre 5

Module loader.py — Chargement et contrôle qualité

I Fonctions principales

Fonction	Description
<code>load_raw(filepath)</code>	Charge le .dta avec pyreadstat. Journalise dimensions et colonnes.
<code>clean_types(df, qc_log)</code>	Normalise les types : codes string, numériques, binaires.
<code>quality_checks(df, qc_log)</code>	Applique outliers, skip patterns, taux de manquants.
<code>run(qc_log)</code>	Point d'entrée : <code>load_raw</code> → <code>clean_types</code> → <code>quality_checks</code> .
<code>_flag_outlier(...)</code>	Met à NaN les valeurs hors bornes, journalise dans <code>qc_log</code> .
<code>_flag_incoherence(...)</code>	Journalise les incohérences sans modifier les données.

TABLE 5.1 – Fonctions de loader.py

II Contrôles de cohérence

Contrôle	Action
Outliers âge répondant (I.3)	Valeurs < 15 ou $> 110 \rightarrow \text{NaN}$
Outliers âge CM (I.9)	Valeurs < 15 ou $> 110 \rightarrow \text{NaN}$
Outliers taille ménage (I.5)	Valeurs < 0 ou $> 50 \rightarrow \text{NaN}$
Skip pattern benne (II.13=2)	Si sans benne, détail II.14–II.20 ne devrait pas être renseigné
Payant sans montant	Si II.24=1 mais II.25_1 manquant $\rightarrow$ alerte
Revente incohérente	Si II.8=2 mais $_v18_v22 = 1 \rightarrow$ alerte
Dépôts sauvages skip	Si III.12=3 mais III.13/14 renseignés $\rightarrow$ alerte
Taux de manquants	Alerte si une variable dépasse 50% de valeurs manquantes

TABLE 5.2 – Contrôles de cohérence appliqués

III Code source

```

1 import os, logging
2 import numpy as np
3 import pandas as pd
4 import pyreadstat
5 from config import (INPUT_DIR, INPUT_FILE, QC_AGE_MIN,
6                     QC_AGE_MAX, QC_TAILLE_MAX, QC_MISSING_THRESH)
7
8 logger = logging.getLogger(__name__)
9
10 def _to_numeric_or_nan(series):
11     return pd.to_numeric(series, errors="coerce")
12
13 def _flag_outlier(df, col, lo, hi, label, qc_log):
14     if col not in df.columns:
15         return df
16     mask = df[col].notna() & ((df[col] < lo) | (df[col] > hi))
17     n = mask.sum()
18     if n > 0:
19         qc_log.append({"check": f"outlier_{col}",
20                      "description": f"{label} : hors [{lo}, {hi}]",
21                      "n_affected": int(n), "action": "Remplace par NaN"})

```

```

22     df.loc[mask, col] = np.nan
23     return df
24
25 def _flag_incoherence(df, mask, label, action, qc_log):
26     n = mask.sum()
27     if n > 0:
28         qc_log.append({"check": "incoherence",
29                       "description": label, "n_affected": int(n),
30                       "action": action})
31     return n
32
33 def load_raw(filepath):
34     logger.info(f"Chargement : {filepath}")
35     df, meta = pyreadstat.read_dta(filepath)
36     logger.info(f"Dimensions : {df.shape[0]} x {df.shape[1]}")
37     return df, meta
38
39 def clean_types(df, qc_log):
40     df = df.copy()
41     str_cat = ["I_4", "I_7", "I_10", "II_3", "II_5", "II_22",
42               "II_23", "III_4", "III_23", "III_26", "III_30", "Commune"]
43     for col in str_cat:
44         if col in df.columns:
45             df[col] = df[col].astype(str).str.strip().replace(
46                 {"": np.nan, "nan": np.nan})
47     num_cols = ["I_3", "I_9", "I_5_1", "I_5_2", "I_5_3", "I_5_4"]
48     for col in num_cols:
49         if col in df.columns:
50             df[col] = _to_numeric_or_nan(df[col])
51     likert = ["I_2", "I_6", "I_8", "I_11", "II_4", "II_6", "II_7",
52              "II_8", "II_13", "II_14", "II_15", "II_16", "II_17",
53              "II_19", "II_20", "II_24", "II_25_1", "II_25_2"]
54     for col in likert:
55         if col in df.columns:
56             df[col] = _to_numeric_or_nan(df[col])
57     for col in [c for c in df.columns if c.startswith("_v")]:
58         df[col] = _to_numeric_or_nan(df[col])
59     logger.info("Nettoyage des types termine.")
60     return df
61
62 def quality_checks(df, qc_log):
63     df = df.copy()
64     df = _flag_outlier(df, "I_3", QC_AGE_MIN, QC_AGE_MAX, "Age repondant",
65                       qc_log)
65     df = _flag_outlier(df, "I_9", QC_AGE_MIN, QC_AGE_MAX, "Age CM", qc_log)

```

```

66     df = _flag_outlier(df, "I_5", 0, QC_TAILLE_MAX, "Taille menage", qc_log
67     )
68     if "II_13" in df.columns:
69         no_benne = df["II_13"] == 2
70         for col in ["II_14", "II_16", "II_17", "II_18", "II_19", "II_20"]:
71             if col in df.columns:
72                 _flag_incoherence(df, no_benne & df[col].notna(),
73                 f"Sans benne mais {col} renseigne",
74                 "Conserve -- skip pattern", qc_log)
75     if "II_24" in df.columns and "II_25_1" in df.columns:
76         _flag_incoherence(df, (df["II_24"]==1) & df["II_25_1"].isna(),
77         "Payant mais montant manquant", "NaN conserve", qc_log)
78     for col in df.columns:
79         r = df[col].isna().mean()
80         if r > QC_MISSING_THRESH:
81             qc_log.append({"check": "missing_rate",
82             "description": f"'{col}' : {r:.1%} manquants",
83             "n_affected": int(df[col].isna().sum()),
84             "action": "Alerte -- verifier skip pattern"})
85     logger.info(f"Controles : {len(qc_log)} signalements.")
86     return df
87
88 def run(qc_log):
89     filepath = os.path.join(INPUT_DIR, INPUT_FILE)
90     df, meta = load_raw(filepath)
91     df = clean_types(df, qc_log)
92     df = quality_checks(df, qc_log)
93     return df, meta

```

Listing 5.1 – loader.py

Chapitre 6

Module builder.py — Construction des variables

I Helpers internes

```
1 def _col(df, col, default=None):
2     """Acces securise : NaN + warning si colonne absente."""
3     if col in df.columns:
4         return df[col]
5     logger.warning(f"Colonne absente : '{col}'")
6     return pd.Series(default if default is not None
7                       else np.nan, index=df.index)
8
9 def _map(series, mapping):
10     return series.map(mapping)
11
12 def _oui_non(series):
13     return series.map({1: "Oui", 2: "Non"})
14
15 def _binary_flag(series):
16     return series.fillna(0).astype(int)
17
18 def _concat_flags(df, cols_labels, sep=" | "):
19     """Liste textuelle des modalites selectionnees."""
20     avail = [(c,l) for c,l in cols_labels if c in df.columns]
21     if not avail:
22         return pd.Series(np.nan, index=df.index)
23     def row_labels(row):
24         lbls = [l for c,l in avail if row.get(c,0) == 1]
25         return sep.join(lbls) if lbls else np.nan
26     return df[[c for c,_ in avail]].apply(row_labels, axis=1)
27
28 def _sum_flags(df, cols):
29     return sum(_col(df,c).fillna(0) for c in cols).astype(int)
```

Listing 6.1 – Helpers builder.py

II Bloc A — Ménage (12 variables)

Variable	Source	Description
<code>commune_code</code>	Commune	Code commune
<code>quartier</code>	quartier	Quartier de l'enquête
<code>zone</code>	—	Constante : Région de Dakar (urbain)
<code>nb_enfants_0_4</code>	I_5_1	Enfants < 5 ans
<code>nb_enfants_5_15</code>	I_5_2	Enfants 5–15 ans
<code>nb_adultes_H</code>	I_5_3	Hommes adultes (15+)
<code>nb_adultes_F</code>	I_5_4	Femmes adultes (15+)
<code>taille_menage</code>	I_5	Taille totale
<code>statut_occupation</code>	I_6	Locataire / Propriétaire / Logé
<code>type_habitat</code>	I_7	Case / Maison / Appartement
<code>proxy_niveau_vie</code>	I_6, I_7, II_24	Score 0–3
<code>proxy_niveau_vie_label</code>	score	Faible / Moyen-bas / Moyen-haut / Élevé

TABLE 6.1 – Variables du Bloc A

Le score proxy est construit comme suit :

$$\text{score} = \mathbf{1}[I_6 = 2] + \mathbf{1}[I_7 \in \{2, 3, 4\}] + \mathbf{1}[II_24 = 1]$$

III Bloc B — Chef de ménage (19 variables)

Variable	Source	Description
repondant_sexe	I_2	Sexe du répondant
repondant_age	I_3	Âge (98/99 → NaN)
repondant_statut	I_4	Lien avec le CM
repondant_est_cm	I_4	Répondant = CM ?
cm_sexe	I_8	Sexe du CM
cm_age	I_9	Âge CM (98/99 → NaN)
cm_tranche_age	I_9	<25 / 25-34 / 35-44 / 45-59 / 60+
cm_type_enseignement	I_10	Française / Arabe / Coranique / Aucun
cm_niveau_instruction	I_11	Primaire / Secondaire / Supérieur
cm_alpha_francais	I_12_2	Alphabétisé en français
cm_alpha_anglais	I_12_3	Alphabétisé en anglais
cm_alpha_arabe	I_12_4	Alphabétisé en arabe
cm_alpha_lng_nat	I_12_5	Alphabétisé en langue nationale
cm_est_alphabetise	I_12_2-5	Au moins une langue
cm_situation_matr.	—	NaN — non collecté
cm_statut_emploi	—	NaN — non collecté
cm_secteur_emploi	—	NaN — non collecté
cm_revenu	—	NaN — non collecté

TABLE 6.2 – Variables du Bloc B

IV Bloc C — Déchets ménagers (87 variables)

IV.1 Sources et natures (II.1–II.2)

- `src__v1` à `src__v7` : indicateurs binaires 0/1 par source
- `sources_liste` : liste textuelle (ex. « Cuisine | Élevage »)
- `nb_sources` : nombre total de sources
- `nat__v8` à `nat__v17` : indicateurs par nature
- `natures_liste`, `nb_natures`

IV.2 Stockage (II.3–II.6)

- `mode_stockage` : Poubelle / Seau / Fût / Carton / Sac
- `stockage_couvert`, `place_stockage`, `capacite_stockage_suffisante`

IV.3 Tri et revente (II.7–II.10)

- `tri_avant_evacuation`, `revente_dechets`, `types_revendus_liste`, `raison_non_revente`

IV.4 Traitement (II.11–II.12)

- `proportion_traitee` : totalité / partie / aucune
- `traitement_enfouissement`, `traitement_incineration`, `traitement_recyclage`, `traitement_compostage`
- `au_moins_un_traitement` : indicateur synthétique (Oui/Non)

IV.5 Bennes tasseuses — service public (II.13–II.20)

- `acces_benne_tasseuse`, `benne_usage_regulier`
- `distance_point_collecte`, `benne_point_nettoyage`, `benne_heure_convient`
- `frequence_benne`, `satisfaction_benne`

IV.6 Service alternatif — entités privées (II.21–II.23)

- `service_alternatif_type` : Charrettes / Pré-collecte / Associations
- `service_alternatif_raison`, `a_service_alternatif`

IV.7 Paiement (II.23–II.25)

- `service_evacuation_payant` : consentement à payer (Oui/Non)
- `montant_evacuation_tranche`, `frequence_paiement`
- `montant_mensuel_evacuation_fcfa` : proxy (0 si gratuit)

IV.8 Infrastructure quartier (III.1–III.33)

- `acces_depotoire_normalise` : bac OU corbeille présent (synthétique)
- `existence_bacs_quartier`, `satisfaction_bacs`, `corbeilles_rue_presentes`

- `dep_sauvages_rue_freq`, `depots_sauvages_observes`
- `nuisible__v23` à `nuisible__v29` : moustiques, mouches, cafards, souris, vers, rats
- `nuisibles_liste`, `nb_nuisibles`
- `service_balayage_rues`, `set_setal`
- `place_publicque`, `gestionnaire_place_publicque`, `salubrite_place_publicque`
- `marche_present`, `gestionnaire_marche`, `gestion_dechets_marche`
- `connait_UCG`, `campagne_sensibilisation`, `satisfait_campagne`
- `suggestions_amelioration` (texte libre)

V Bloc D — Conséquences sanitaires (14 variables)

Variable	Source	Description
<code>maladie__v30</code>	<code>__v30</code>	Fièvre (0/1)
<code>maladie__v31</code>	<code>__v31</code>	Asthme (0/1)
<code>maladie__v32</code>	<code>__v32</code>	Rhume (0/1)
<code>maladie__v33</code>	<code>__v33</code>	Sinusite (0/1)
<code>maladie__v34</code>	<code>__v34</code>	Toux (0/1)
<code>maladie__v35</code>	<code>__v35</code>	Nausées/Vomissements (0/1)
<code>maladie__v36</code>	<code>__v36</code>	Démangeaisons (0/1)
<code>maladie__v37</code>	<code>__v37</code>	Aucune maladie (0/1)
<code>maladie__v38</code>	<code>__v38</code>	Autre maladie (0/1)
<code>maladies_liste</code>	<code>__v30–38</code>	Liste textuelle
<code>nb_maladies</code>	<code>__v30–36,38</code>	Nombre de types (hors aucune)
<code>aucune_maladie</code>	<code>__v37</code>	Indicateur 0/1
<code>nb_types_maladies</code>	<code>IV_count</code>	Nombre total dans le ménage
<code>maladie_liee_dechets</code>	<code>__v31,33–36</code>	Oui si asthme/sinusite/toux/nausées/démangeaisons

TABLE 6.3 – Variables du Bloc D

Note

L'indicateur `maladie_liee_dechets` est :

$$\mathbf{1} \left[\sum_{j \in \{31, 33, 34, 35, 36\}} -v_j \geq 1 \right]$$

La fièvre et le rhume sont exclus car non spécifiques aux déchets.

Chapitre 7

Modules exporter.py et qaqc.py

I exporter.py

Fonction	Description
<code>_prepare_for_stata(df)</code>	Raccourcit les noms > 32 caractères (limite Stata).
<code>export_csv(df, filename)</code>	Export CSV UTF-8 avec BOM (<code>utf-8-sig</code>) pour Excel.
<code>export_dta(df, filename)</code>	Export Stata .dta avec labels de variables.
<code>run(df_final)</code>	Appelle les deux exports, retourne les chemins.

TABLE 7.1 – Fonctions de exporter.py

Note

Stata impose une limite de **32 caractères** sur les noms de variables. Exemple : `satisfaction_emplacements_corbeilles` → `satisfaction_emplacements_cor`

II qaqc.py — Rapport Excel 6 onglets

Onglet	Contenu
01_Résumé	Dimensions brutes vs traitées, taux de complétion, variables absentes
02_Anomalies_QC	Journal des 49 anomalies : type, description, N obs., action
03_Manquants	Taux par variable, trié décroissant, code couleur
04_Estimations_Primaires	Distributions de toutes les variables clés par bloc
05_Montant_Evacuation	Stats du proxy montant mensuel (moyenne, médiane, min, max)
06_Variables_Absentes	Documentation des 9 variables non collectées

TABLE 7.2 – Structure du rapport QAQC

Chapitre 8

Orchestrateur run_pipeline.py

```
1  #!/usr/bin/env python3
2  import os, sys, logging, time
3  from datetime import datetime
4
5  sys.path.insert(0, os.path.dirname(__file__))
6  import loader, builder, qaqc, exporter
7  from config import OUTPUT_DIR
8
9  os.makedirs(OUTPUT_DIR, exist_ok=True)
10 log_file = os.path.join(OUTPUT_DIR,
11     f"pipeline_{datetime.now().strftime('%Y%m%d_%H%M%S')}.log")
12 logging.basicConfig(level=logging.INFO,
13     format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
14     handlers=[logging.StreamHandler(sys.stdout),
15         logging.FileHandler(log_file, encoding="utf-8")])
16 logger = logging.getLogger("pipeline")
17
18 def run():
19     start = time.time()
20     logger.info("PIPELINE DECHETS MENAGERS -- DEMARRAGE")
21     qc_log = []
22     logger.info("[ETAPE 1/4] Chargement et controle qualite...")
23     df_raw, meta = loader.run(qc_log)
24     logger.info("[ETAPE 2/4] Construction des variables...")
25     df_final = builder.run(df_raw)
26     logger.info("[ETAPE 3/4] Export CSV et Stata...")
27     exporter.run(df_final)
28     logger.info("[ETAPE 4/4] Rapport QAQC...")
29     qaqc_path = os.path.join(OUTPUT_DIR, "QAQC_rapport.xlsx")
30     qaqc.run(df_raw, df_final, qc_log, qaqc_path)
31     elapsed = round(time.time() - start, 1)
32     logger.info(f"TERMINE -- {elapsed}s | "
33         f"{df_final.shape[0]} menages | "
34         f"{df_final.shape[1]} variables | "
35         f"{len(qc_log)} anomalies")
36     return df_final, qc_log
37
```

```
38 if __name__ == "__main__":  
39     run()
```

Listing 8.1 – run_pipeline.py

Chapitre 9

API FastAPI — main.py

I Routes

Méthode	Route	Description
GET	/	Interface web HTML
POST	/api/process	Reçoit .dta, lance pipeline, retourne ZIP
GET	/api/info	Informations JSON sur le pipeline
GET	/docs	Documentation Swagger

TABLE 9.1 – Routes de l'API

II Flux POST /api/process

1. Validation : vérifier l'extension `.dta`
2. Création d'un répertoire temporaire isolé
3. Sauvegarde du fichier uploadé
4. Patch dynamique de `config.INPUT_DIR` et `config.OUTPUT_DIR`
5. Exécution : loader → builder → export → qaqc
6. Compression dans un ZIP en mémoire (BytesIO)
7. Retour via `StreamingResponse`

III Code source

```
1 import io, os, sys, tempfile, zipfile, logging
2 from pathlib import Path
3 from fastapi import FastAPI, File, UploadFile, HTTPException
4 from fastapi.responses import HTMLResponse, StreamingResponse,
  JSONResponse
```

```

5 from fastapi.templating import Jinja2Templates
6 from fastapi.requests import Request
7
8 sys.path.insert(0, os.path.join(os.path.dirname(__file__), "src"))
9 app = FastAPI(title="Pipeline Dechets Menagers", version="1.0.0")
10 templates = Jinja2Templates(directory="templates")
11
12 @app.get("/", response_class=HTMLResponse)
13 async def index(request: Request):
14     return templates.TemplateResponse("index.html", {"request":
15         request})
16
17 @app.post("/api/process")
18 async def process_file(file: UploadFile = File(...)):
19     if not file.filename.endswith(".dta"):
20         raise HTTPException(400, "Format invalide : .dta requis.")
21     tmp = Path(tempfile.mkdtemp())
22     (tmp/"input").mkdir(); (tmp/"output").mkdir()
23     (tmp/"input"/"Base.dta").write_bytes(await file.read())
24     try:
25         import importlib, config, loader, builder, qaqc, exporter
26         config.INPUT_DIR = str(tmp/"input")
27         config.OUTPUT_DIR = str(tmp/"output")
28         config.INPUT_FILE = "Base.dta"
29         importlib.reload(loader); importlib.reload(exporter)
30         qc_log = []
31         df_raw, _ = loader.run(qc_log)
32         df_final = builder.run(df_raw)
33         csv = str(tmp/"output"/"dataset_dechets.csv")
34         dta = str(tmp/"output"/"dataset_dechets.dta")
35         xlsx = str(tmp/"output"/"QAQC_rapport.xlsx")
36         df_final.to_csv(csv, index=False, encoding="utf-8-sig")
37         import pyreadstat
38         df_s, _ = exporter._prepare_for_stata(df_final)
39         pyreadstat.write_dta(df_s, dta)
40         qaqc.run(df_raw, df_final, qc_log, xlsx)
41     except Exception as e:
42         raise HTTPException(500, str(e))
43     buf = io.BytesIO()
44     with zipfile.ZipFile(buf, "w", zipfile.ZIP_DEFLATED) as zf:
45         zf.write(csv, "dataset_dechets.csv")
46         zf.write(dta, "dataset_dechets.dta")
47         zf.write(xlsx, "QAQC_rapport.xlsx")
48     buf.seek(0)
49     return StreamingResponse(buf, media_type="application/zip",

```

```
49     headers={"Content-Disposition":
50             "attachment; filename=resultats_pipeline.zip"})
51
52 @app.get("/api/info")
53 async def info():
54     return JSONResponse({"pipeline": "ENSAE/UCG 2018", "version": "1.0.0"
55                          ,
56                          "format": ".dta", "variables": 132, "menages": 2474})
57
58 if __name__ == "__main__":
59     import uvicorn
60     uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

Listing 9.1 – main.py

Troisième partie

VARIABLES, UTILISATION ET DÉ- PLOIEMENT

Chapitre 10

Variables produites et absentes

I Récapitulatif

Bloc	Nb	Thème
A — Ménage	12	Géographie, habitat, proxy niveau de vie
B — Chef de ménage	19	Démographie, instruction, alphabétisation
C — Déchets	87	Sources, stockage, évacuation, infrastructure
D — Conséquences	14	Maladies déclarées, indicateurs sanitaires
Total	132	

TABLE 10.1 – Variables analytiques produites

II Variables absentes

Variable	Statut	Note
<code>region</code>	ABSENT	Zone entièrement urbaine (Dakar).
<code>departement</code>	ABSENT	Non extrait comme variable séparée.
<code>milieu_residence</code>	ABSENT	Toute la zone est urbaine.
<code>depenses_menage</code>	PROXY	Score proxy via I_6 + I_7 + II_24.
<code>actifs_menage</code>	PROXY	Inclus dans proxy_niveau_vie (0–3).
<code>revenu_CM</code>	PROXY	Proxy via statut et paiement déchets.
<code>situation_matr._CM</code>	ABSENT	Non collectée dans ce questionnaire.
<code>statut_emploi_CM</code>	ABSENT	Non collecté.
<code>secteur_emploi_CM</code>	ABSENT	Non collecté.

TABLE 10.2 – Variables absentes et documentation

Chapitre 11

Guide d'utilisation et déploiement

I Installation

```
1 # Dézipper le projet
2 cd Groupe_Insalubrite_traitement_donnees
3
4 # Installer les dépendances
5 pip install -r requirements.txt
6
7 # Copier Base.dta dans input/
```

Listing 11.1 – Installation

II Option A — Ligne de commande

```
1 cd src
2 python run_pipeline.py
3 # Resultats dans output/
```

Listing 11.2 – Pipeline en ligne de commande

III Option B — Plateforme web locale

```
1 # Depuis la racine du projet
2 python -m uvicorn main:app --reload
3 # Ouvrir http://127.0.0.1:8000
```

Listing 11.3 – Plateforme web

IV Déploiement sur Render

```

1 git init && git add .
2 git commit -m "pipeline dechets v1.0"
3 git push origin main
4 # Sur render.com : New > Web Service > connecter le repo
5 # Render lit render.yaml automatiquement

```

Listing 11.4 – Deploiement

```

services:
  - type: web
    name: pipeline-dechets-menagers
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
    envVars:
      - key: PYTHON_VERSION
        value: 3.11.0

```

Listing 11.5 – render.yaml

V Dépendances Python

Package	Version	Usage
fastapi	0.115.0	Framework API REST
uvicorn[standard]	0.30.6	Serveur ASGI
python-multipart	0.0.9	Gestion uploads
jinja2	3.1.4	Templates HTML
pyreadstat	1.3.1	Lecture/écriture Stata
pandas	2.2.2	Manipulation DataFrames
numpy	1.26.4	Calculs numériques
openpyxl	3.1.2	Génération Excel

TABLE 11.1 – requirements.txt

1. Placer le nouveau `.dta` dans `input/`
2. Vérifier les codes dans `src/config.py`
3. Relancer : `python src/run_pipeline.py`
4. Aucune modification des autres modules n'est nécessaire